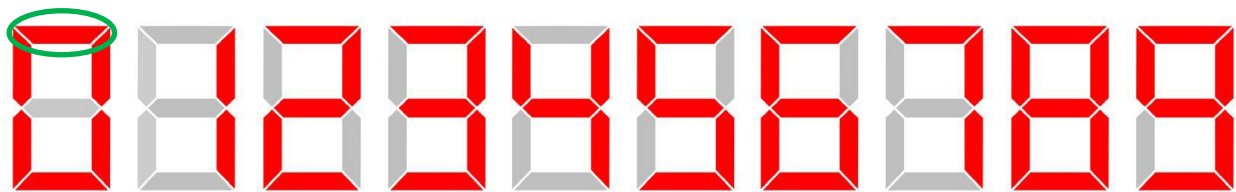


Laborübung 3

In dieser Laborübung sind 2 Aufgaben zu lösen, eine behandelt das Thema Optimierung durch KV-Diagramme (Aufgabe 1 + Zusatz), die andere behandelt konkurrierende Prozesse und deren Auswirkungen (Aufgabe 2).

Aufgabe 1: KV-Diagramm

Ein (!!!) Segment (grüne Ellipse) einer 7-Segment LCD-Anzeige zur Darstellung von Ziffern soll mit einer Schaltung angesteuert werden. Die zehn möglichen Ziffern (siehe Abbildung) sind mittels BCD-Code codiert. Sollte (im Fehlerfall) eine der eigentlich ungenutzten sechs Pseudo-Tetraden anliegen, sollen alle Segmente ausgeschaltet sein.



Erstellen Sie die Wahrheitstabelle, die DNF und minimieren Sie diese mittels KV-Diagramm. Erstellen Sie anschließend eine Schaltung mit Logikgattern für das Segment (handschriftlich oder im Simulator).

Zusatzaufgabe:

Wir verzichten im Folgenden auf die Forderung, dass alle Segmente ausgeschaltet sein sollen, wenn eine der eigentlich ungenutzten sechs Pseudo-Tetraden anliegt. Das heißt, dass uns für diese Pseudo-Tetraden der Wert am Ausgang unserer Schaltung völlig egal ist („don’t care“) und dass Sie für jede einzelne dieser sechs Pseudo-Tetraden nach eigenem Ermessen festlegen können, ob diese am Ausgang 0 oder 1 liefern. Lässt sich mit diesen zusätzlichen Freiheiten die Ansteuerung noch weiter vereinfachen? Versuchen Sie, in Ihrem unter 2. erzeugten KV-Diagramm mit Hilfe der „don’t care“-Felder größere Blöcke zu bilden. Notieren Sie die resultierende minimale DNF. Erstellen Sie anschließend eine Schaltung mit Logikgattern für das Segment.

Aufgabe 2: konkurrierende Prozesse

Zur Veranschaulichung von **konkurrierenden Prozessen** sollen 2 Prozesse (session1 und session2) innerhalb eines Datenbanksystems untersucht werden.

Insgesamt werden 5 (Anweisungs-)Blöcke nacheinander abgearbeitet. Für jeden dieser Blöcke gibt es ein entsprechendes Video (d.h. Sie führen selbst keine Anweisungen auf dem Datenbanksystem durch, sondern interpretieren das in den Videos gezeigte Geschehen).

Für jeden Block soll ein (Teil-)Report erstellt werden. Dieser soll eine zweispaltige Tabelle (eine für session 1, eine für session2) mit dem zugehörigen Verhalten/Output der beiden Sessions und eventuelle Interpretationen/Kommentare Ihrerseits beinhalten. Bitte stellen Sie sicher, dass die zeitliche Ordnung klar erkennbar ist. Eine ziemlich einfache Beispiel-Tabelle sehen Sie hier:

	Session1	Session2
Block Nr X	Es wurde dieses und jenes gemacht und die Ausgabe ist: Nun wurde jenes gemacht	Hier ist nichts zu sehen Jetzt ist auch hier etwas zu sehen, nämlich:
Schlussfolgerung:		
Block Nr X+1		

Im Folgenden sind die Anweisungen notiert, die in den Videos durchgeführt werden:

Block A:

Eine neue Tabelle wird in *Session1* erzeugt:

```
CREATE TABLE ttrans (
  id NUMBER(5) NOT NULL CONSTRAINT pk_ttrans PRIMARY KEY,
  n  NUMBER(5));
```

Wann ist die Tabelle in *Session2* sichtbar?

```
DESCRIBE ttrans;
```

Block B:

Es werden die folgenden Datensätze in *Session1* in die in Block1 erstellte Tabelle eingefügt:

```
INSERT INTO ttrans VALUES (1,1);
INSERT INTO ttrans VALUES (2,2);
INSERT INTO ttrans VALUES (3,3);
```

Anzeigen der Tabelleninhalt in *beiden* Sessions:

```
SELECT * FROM ttrans;
```

Die Transaktion wird in *Session1* abgeschlossen:

```
COMMIT;
```

Erneutes Anzeigen der Tabelleninhalte in *beiden* Sessions:

```
SELECT * FROM ttrans;
```

Welche Schlussfolgerung ziehen Sie aus Ihren Beobachtungen?

Block C:

In *Session1* wird im Datensatz mit der id=3 der Wert von n auf 33 abgeändert:

```
UPDATE ttrans SET n=33 WHERE id=3;
```

Der Datensatz mit der id=3 wird in *beiden* Sessions angezeigt:

```
SELECT * FROM ttrans WHERE id=3;
```

Die Transaktion in *Session1* wird wieder „zurück gerollt“:

```
ROLLBACK;
```

Erneutes Anzeigen des Datensatzes mit der id=3 in *beiden* Sessions:

```
SELECT * FROM ttrans WHERE id=3;
```

Welche Schlussfolgerung ziehen Sie aus Ihren Beobachtungen?

Block D:

In *Session1* wird im Datensatz mit der *id=1* der Wert von *n* mit 2 multipliziert:

```
UPDATE ttrans SET n=n*2 WHERE id=1;
```

Welchen Output liefern *Session1* bzw. *Session2*?

In *Session2* wird im gleichen Datensatz (*id=1*) der Wert von *n* mit 3 multipliziert:

```
UPDATE ttrans SET n=n*3 WHERE id=1;
```

Welchen Output liefern *Session1* bzw. *Session2* und warum?

(Script Runner Task ist keine gültige Antwort, da das nur eine Info vom Tool ist)

Die Transaktion in *Session1* wird abgeschlossen:

```
COMMIT;
```

Welchen Output liefern *Session1* bzw. *Session2*?

Die Transaktion in *Session2* wird abgeschlossen:

```
COMMIT;
```

Welchen Output liefern *Session1* bzw. *Session2*?

Der Datensatz mit der *id=1* wird in *beiden* Sessions angezeigt:

```
SELECT * FROM ttrans WHERE id=1;
```

Die Operation welcher Session war erfolgreich? Welche Schlussfolgerung ziehen Sie aus Ihren Beobachtungen?

Block E:

Es wird eine Deadlock-Situation zwischen *Session1* und *Session2* provoziert.

Das geschieht bspw. folgendermaßen:

Session1: UPDATE ttrans SET n=11 WHERE id=1;

Session2: UPDATE ttrans SET n=22 WHERE id=2;

Session1: UPDATE ttrans SET n=222 WHERE id=2;

Session2: UPDATE ttrans SET n=111 WHERE id=1;

Wie reagiert das System?

In welchem Zustand sind die beiden Sessions?

Wie würden Sie in *session1* bzw. *session2* weiter vorgehen (aus den Erfahrungen der bisherigen Blöcke), damit alles ein gutes Ende nimmt?

Beachten Sie dabei, dass es sich um lange Scripte handeln könnte (nicht nur die beiden beispielhaften Befehle pro Session) und dass die konkurrierenden Parteien nichts voneinander wissen (eine läuft bspw. in Hamburg, die andere in New York).